

Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc", "car", "ada", "racecar", "cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome", "racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

Program :

```
#include <stdio.h>

#include <string.h>

int isPalindrome(char word[]){

    int i, len;

    len = strlen(word);

    for (i = 0; i < len / 2; i++){

        if (word[i] != word[len - i - 1]){

            return 0;

        }

    }

    return 1;

}

int main(){

    int n, i;

    char words[100][100];

    printf("Enter number of strings: ");

    scanf("%d", &n);

    printf("Enter the strings:\n");

    for (i = 0; i < n; i++){

        scanf("%s", words[i]);

    }

    for (i = 0; i < n; i++){
```

```

    if (isPalindrome(words[i])){
        printf("First palindrome: %s", words[i]);
        return 0;
    }
}

printf("No palindrome found");
return 0;
}

```

Input:

```

Enter number of strings: 4
Enter the strings:
car ada lol bike

```

Output:

```

First palindrome: ada

```

2.You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively. Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1`. Return `[answer1,answer2]`.

Example 1:

Input: `nums1 = [2,3,2]`, `nums2 = [1,2]`

Output: `[2,1]`

Explanation:

Example 2:

Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]`

Output: `[3,4]`

Explanation:

The elements at indices 1, 2, and 3 in `nums1` exist in `nums2` as well. So `answer1` is 3.

The elements at indices 0, 1, 3, and 4 in `nums2` exist in `nums1`. So `answer2` is 4.

```

#include <stdio.h>

```

```

int exists(int arr[], int size, int value) {

```

```

    for (int i = 0; i < size; i++)

```

```

        if (arr[i] == value)

```

```

            return 1;

```

```

    return 0;

```

```

}

int main() {

    int nums1[100], nums2[100];

    int n, m;

    int answer1 = 0, answer2 = 0;

    printf("Enter size of nums1: ");

    scanf("%d", &n);

    printf("Enter %d elements of nums1: ", n);

    for (int i = 0; i < n; i++)

        scanf("%d", &nums1[i]);

    printf("Enter size of nums2: ");

    scanf("%d", &m);

    printf("Enter %d elements of nums2: ", m);

    for (int i = 0; i < m; i++)

        scanf("%d", &nums2[i]);

    for (int i = 0; i < n; i++)

        if (exists(nums2, m, nums1[i]))

            answer1++;

    for (int i = 0; i < m; i++)

        if (exists(nums1, n, nums2[i]))

            answer2++;

    printf("Output: [%d,%d]\n", answer1, answer2);

    return 0;

}

```

Input :

```

Enter size of nums1: 3
Enter 3 elements of nums1: 2 2 3
Enter size of nums2: 2
Enter 2 elements of nums2: 1 2

```

Output :

```

Output: [2,1]

```

3. You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

`[1]`: 1 distinct value

`[2]`: 1 distinct value

`[1]`: 1 distinct value

`[1,2]`: 2 distinct values

`[2,1]`: 2 distinct values

`[1,2,1]`: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2:

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

`[1]`: 1 distinct value

`[1]`: 1 distinct value

`[1,1]`: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

```
#include <stdio.h>
```

```
int main() {
```

```
    int nums[100];
```

```
    int n, total = 0;
```

```
    printf("Enter size of array: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d elements: ", n);
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &nums[i]);
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = i; j < n; j++) {
```

```
            // count distinct values in nums[i..j]
```

```
            int distinct = 0;
```

```
            for (int k = i; k <= j; k++) {
```

```
                int seen = 0;
```

```
                for (int p = i; p < k; p++) {
```

```
                    if (nums[p] == nums[k]) {
```

```
                        seen = 1;
```

```

        break;
    }
}
if (!seen)
    distinct++;
}
total += distinct * distinct;
}
}

printf("Output: %d\n", total);

return 0;
}

```

Input :

```

Enter size of array: 3
Enter 3 elements: 1 2 1

```

Output :

```

Output: 15

```

4. Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return *the number of pairs* (i, j) *where* $0 \leq i < j < n$, *such that* `nums[i] == nums[j]` *and* $(i * j)$ *is divisible by* `k`.

Example 1:

Input: `nums = [3,1,2,2,2,1,3]`, `k = 2`

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- `nums[0] == nums[6]`, and $0 * 6 == 0$, which is divisible by 2.
- `nums[2] == nums[3]`, and $2 * 3 == 6$, which is divisible by 2.
- `nums[2] == nums[4]`, and $2 * 4 == 8$, which is divisible by 2.
- `nums[3] == nums[4]`, and $3 * 4 == 12$, which is divisible by 2.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 1`

Output: 0

Explanation: Since no value in `nums` is repeated, there are no pairs (i, j) that meet all the requirements.

```
#include <stdio.h>
```

```
int main() {
```

```
    int nums[100];
```

```
    int n, k, count = 0;
```

```
    printf("Enter size of array: ");
```

```

scanf("%d", &n);

printf("Enter %d elements: ", n);

for (int i = 0; i < n; i++)

    scanf("%d", &nums[i]);

printf("Enter k: ");

scanf("%d", &k);

for (int i = 0; i < n; i++) {

    for (int j = i + 1; j < n; j++) {

        if (nums[i] == nums[j] && (i * j) % k == 0)

            count++;

    }

}

printf("Output: %d\n", count);

return 0;
}

```

Input :

```

Enter size of array: 7
Enter 7 elements: 3 1 2 2 2 1 3
Enter k: 2

```

Output :

```

Output: 4

```

5. Write a program FOR THE BELOW TEST CASES with least time complexity

Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

4. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List
 1. Input: []
 2. Expected Output: None or an appropriate message indicating that the list is empty.
2. Single Element List
 1. Input: [5]
 2. Expected Output: 5
3. All Elements are the Same
 1. Input: [3, 3, 3, 3, 3]
 2. Expected Output: 3

```
#include <stdio.h>
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int temp[100];
```

```
    int i = left, j = mid + 1, k = 0;
```

```
    while (i <= mid && j <= right) {
```

```
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
```

```
        else temp[k++] = arr[j++];
```

```
    }
```

```
    while (i <= mid) temp[k++] = arr[i++];
```

```
    while (j <= right) temp[k++] = arr[j++];
```

```
    for (i = 0; i < k; i++)
```

```
        arr[left + i] = temp[i];
```

```
}
```

```
void mergeSort(int arr[], int left, int right) {
```

```
    if (left < right) {
```

```
        int mid = (left + right) / 2;
```

```
        mergeSort(arr, left, mid);
```

```
        mergeSort(arr, mid + 1, right);
```

```
        merge(arr, left, mid, right);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int arr[100];
```

```

int n;

printf("Enter size of list: ");

scanf("%d", &n);

if (n <= 0) {

    printf("List is empty\n");

    return 0;

}

printf("Enter %d elements: ", n);

for (int i = 0; i < n; i++)

    scanf("%d", &arr[i]);

mergeSort(arr, 0, n - 1);

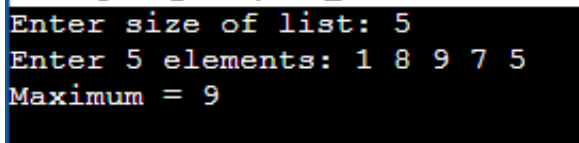
printf("Maximum = %d\n", arr[n - 1]);

return 0;

}

```

Input & Output :



```

Enter size of list: 5
Enter 5 elements: 1 8 9 7 5
Maximum = 9

```

5. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

```
#include <stdio.h>
```



```

int main() {

    int arr[100], unique[100];

    int n, count = 0;

    printf("Enter size of list: ");

    scanf("%d", &n);

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)

        scanf("%d", &arr[i]);

    for (int i = 0; i < n; i++) {

        int duplicate = 0;

        for (int j = 0; j < count; j++) {

            if (unique[j] == arr[i]) {

                duplicate = 1;

                break;

            }

        }

        if (!duplicate)

            unique[count++] = arr[i];

    }

    printf("Unique list: [");

    for (int i = 0; i < count; i++) {

        printf("%d", unique[i]);

        if (i < count - 1) printf(", ");

    }

    printf("]\n");

    return 0;

}

```

Input & Output :

```

Enter size of list: 5
Enter 5 elements: 7 7 8 1 1
Unique list: [7, 8, 1]

```

6. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

```
#include <stdio.h>

int main() {

    int arr[100];

    int n;

    printf("Enter size of array: ");

    scanf("%d", &n);

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)

        scanf("%d", &arr[i]);

    for (int i = 0; i < n - 1; i++) {

        int swapped = 0;

        for (int j = 0; j < n - 1 - i; j++) {

            if (arr[j] > arr[j + 1]) {

                int temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

                swapped = 1;

            }

        }

        if (!swapped)

            break;

    }

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)

        printf("%d ", arr[i]);

    printf("\n");

    return 0;

}
```

Input & Output :

```
Enter size of array: 4
Enter 4 elements: 7 5 6 1
Sorted array: 1 5 6 7
```

7. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

Example $X=\{ 3,4,6,-9,10,8,9,30\}$ KEY=10

Output: Element 10 is found at position 5

Example $X=\{ 3,4,6,-9,10,8,9,30\}$ KEY=100

Output : Element 100 is not found

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int key) {
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
        if (arr[mid] == key)
```

```
            return mid;
```

```
        else if (arr[mid] < key)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int original[100], sorted[100];
```

```
    int n, key;
```

```
    printf("Enter size of array: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d elements: ", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &original[i]);
```

```
        sorted[i] = original[i];
```

```
    }
```

```

printf("Enter key to search: ");

scanf("%d", &key);

for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
        if (sorted[j] > sorted[j + 1]) {
            int t = sorted[j];
            sorted[j] = sorted[j + 1];
            sorted[j + 1] = t;
        }

if (binarySearch(sorted, n, key) == -1) {
    printf("Element %d is not found\n", key);
} else {
    for (int i = 0; i < n; i++)
        if (original[i] == key) {
            printf("Element %d is found at position %d\n", key, i + 1);
            break;
        }
}

return 0;
}

```

Input & Output :

```

Enter size of array: 5
Enter 5 elements: 1 2 3 4 5
Enter key to search: 4
Element 4 is found at position 4

```

8. Given an array of integers `nums`, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

```

#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {

    int temp[100];

    int i = left, j = mid + 1, k = 0;

```

```

while (i <= mid && j <= right) {

    if (arr[i] <= arr[j]) temp[k++] = arr[i++];

    else          temp[k++] = arr[j++];

}

while (i <= mid) temp[k++] = arr[i++];

while (j <= right) temp[k++] = arr[j++];


for (i = 0; i < k; i++)

    arr[left + i] = temp[i];

}

void mergeSort(int arr[], int left, int right) {

    if (left < right) {

        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);

        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);

    }

}

int main() {

    int arr[100];

    int n;

    printf("Enter size of list: ");

    scanf("%d", &n);

    if (n <= 0) {

        printf("List is empty\n");

        return 0;

    }

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)

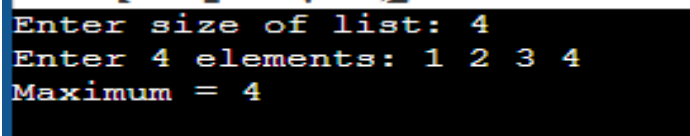
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

```

```
printf("Maximum = %d\n", arr[n - 1]); // last element after sorting  
return 0;  
}
```

Input & Output:



```
Enter size of list: 4  
Enter 4 elements: 1 2 3 4  
Maximum = 4
```